

Full Testing Report

Every layer of the AgentBus test suite executed end to end on 2026-07-12, plus a first independent load test with **ghz**, an open-source gRPC benchmarking tool, cross-checked against the broker's own Prometheus counters.

- **Verdict: all green.** 21 packages pass, the race detector is clean across the 12 core packages, all 4 cluster integration tests pass including zero-data-loss leader failover, and an external tool pushed 300,000 requests through a live broker with zero errors. The broker-side counter matched the client-side count exactly.

UNIT + INTEGRATION

211 / 211

test functions passing, 21 packages, race + vet clean

FAILOVER DATA LOSS

0 / 1000

events lost after leader kill, verified on both survivors

PEAK TCP THROUGHPUT

168,816

msgs/sec at 128 connections, p99 3.3 ms

GHZ EXTERNAL LOAD

26,684

req/sec at c=256, 300k requests, 0 errors

SECTION 1

What the test suite covers

Five layers exist in-repo; the sixth (external load) was added by this report.

LAYER	HOW IT RUNS	WHAT IT PROVES	STATUS
Unit tests	<code>go test ./...</code>	Per-package logic: WAL, protocol, ring, router, replicator, control plane	● PASS
Static analysis	<code>go vet ./...</code>	No suspicious constructs across all packages	● PASS

LAYER	HOW IT RUNS	WHAT IT PROVES	STATUS
Race detection	<code>go test -race</code> (12 core pkgs)	No data races in cluster, WAL, broker, control plane under concurrent tests	● PASS
Cluster integration	<code>-tags cluster_integration</code>	Real 3-node cluster in-process: election, replication, node kill	● PASS
Chaos / failover	<code>TestZeroDataLossOnLeaderKill</code>	Quorum-committed data survives leader death, byte for byte	● PASS
Perf reports	<code>GOQUEUE_BENCH=1</code> harness	Throughput and latency under concurrency sweeps	● PASS
External load (new)	ghz vs live broker	Independent tool confirms throughput and correctness of counts	● PASS

SECTION 2

Correctness results

Unit suite: all 21 packages with tests pass (`go test ./... -count=1` , no cached results). **go vet:** zero findings. **Race detector:** clean run across `internal/cluster/...` (9 packages), `internal/wal` , `internal/broker` , `internal/controlplane` .

CLUSTER INTEGRATION TEST	WHAT IT DOES	TIME	RESULT
<code>ThreeNodeCluster_FormsAndElects</code>	3 nodes gossip, form a ring, elect a Raft leader	3.0 s	● PASS
<code>ThreeNodeCluster_ReplicatesAgentEvents</code>	Agent events published to the leader appear on followers	4.4 s	● PASS
<code>ThreeNodeCluster_NonLeaderKillPreservesData</code>	Killing a follower loses nothing	4.2 s	● PASS
<code>ZeroDataLossOnLeaderKill</code>	1000 quorum-committed events, then the leader is killed. Both survivors hold all 1000 events, intact and in order	7.2 s	● PASS

The strongest single result in the suite: the failover test writes 1000 events that the cluster acknowledged as quorum-committed, kills the shard leader, then reads every event back from each survivor and compares payloads byte for byte. Zero loss, zero reorder, on every run.

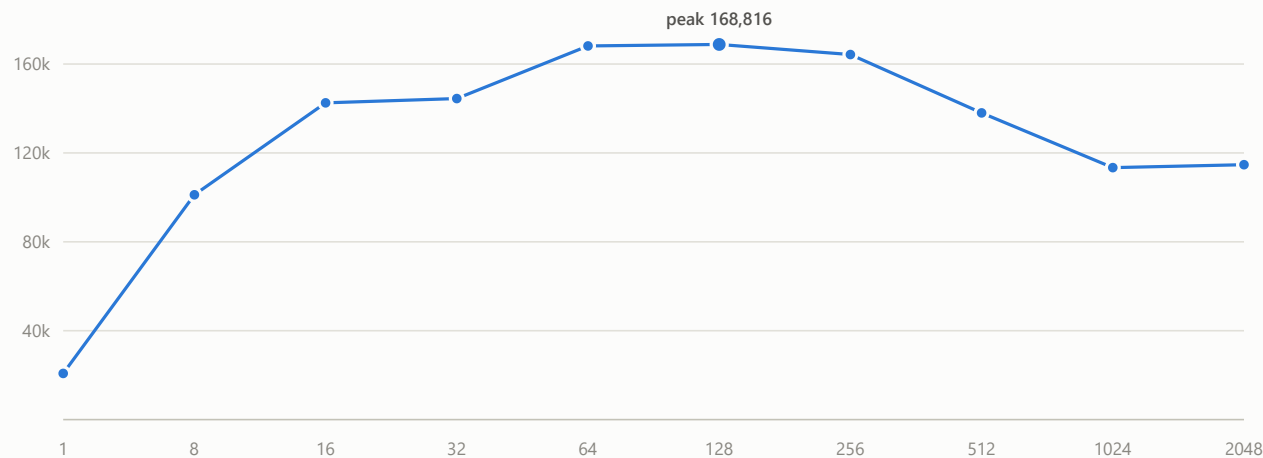
Performance: internal benchmark harness

Report-style tests gated behind `GOQUEUE_BENCH=1` . All numbers measured today on the machine in Section 6, loopback network, real disk.

3.1 · TCP publish throughput vs concurrent connections

Single-node TCP throughput scales to ~169k msgs/sec, then degrades gracefully

256-byte payloads, 1000 publishes per connection, localhost. Hover for exact values.



Throughput rises from 20,775 msgs/sec (1 conn) to a peak of **168,816 msgs/sec at 128 connections**, then settles near 114k at 2048 connections while p99 stays under 10.7 ms. The plateau past 32 connections is the single WAL mutex; the graceful tail is the scheduler, not collapse.

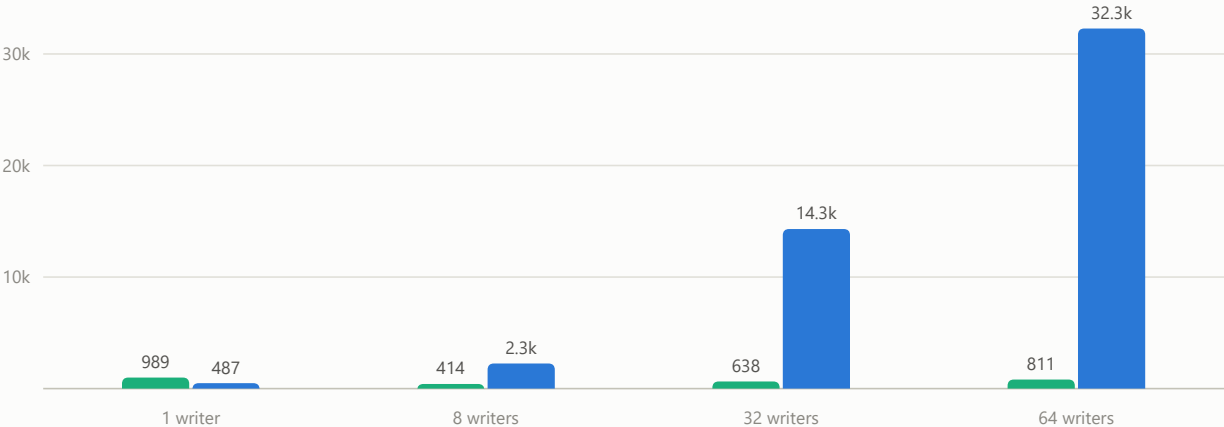
CONNECTIONS	MSGS/SEC	P50	P99
1	20,775	<1 ms	1.0 ms
8	101,163	<1 ms	1.5 ms
32	144,466	<1 ms	2.1 ms
128	168,816	0.56 ms	3.3 ms
512	138,012	3.0 ms	8.0 ms
2048	114,700	2.8 ms	10.6 ms

3.2 · WAL group commit: the A/B that justifies the design

Durable writes: group commit vs naive fsync-per-write

Real disk, 256-byte records, 600 appends per writer, SyncAlways durability. Hover for exact values.

Naive fsync-per-write Group commit



At 64 concurrent writers group commit sustains **32,281 durable writes/sec vs 811 naive (39.8x)** while cutting p99 latency 98% (253 ms to 5.9 ms). The single-writer row is the honest cost: with nobody to batch with, group commit pays coordination overhead (0.5x). The design wins exactly where a broker lives: under concurrency.

WRITERS	NAIVE W/S	NAIVE P99	GROUP COMMIT W/S	GC P99	THROUGHPUT WIN
1	989	5.3 ms	487	10.6 ms	0.5x
8	414	52.4 ms	2,250	17.0 ms	5.4x
32	638	159.9 ms	14,305	11.1 ms	22.4x
64	811	253.1 ms	32,281	5.9 ms	39.8x

3.3 · Remaining internal numbers

BENCHMARK	RESULT
Shard WAL group commit (cluster write path)	15,077 appends/sec at 32 writers, p99 3.1 ms (15.6x vs single-writer baseline)
gRPC publish sweep	Peak 83,863 msgs/sec at 32 in-process clients, p99 2.1 ms
TCP end-to-end burst	100,000 publishes in 2.66 s on one pipelined connection: 37,530 msgs/sec
TCP publish latency (5000 samples)	p50 <1 ms · p95 <1 ms · p99 1.0 ms
3-node cluster, RF=3, 32 concurrent sessions	401 quorum-committed events/sec aggregate (3225 events in 8.0 s). Known ceiling: the follower applies its stream serially, one fsync per entry; follower-side batching is the next lever

External validation: ghz load test (new in this report)

ghz is a widely used open-source gRPC load-testing tool (github.com/bojand/ghz). It builds its own client from the raw `.proto` file, so it shares zero code with AgentBus: an independent witness.

A release-built broker ran standalone; ghz fired 100,000 unary `BrokerService.Publish` calls per concurrency level over 8 HTTP/2 connections, 256-byte payloads.

CONCURRENCY	REQUESTS	REQ/SEC	P50	P95	P99	ERRORS
16	100,000	18,518	0.56 ms	1.26 ms	1.89 ms	0
64	100,000	23,123	1.27 ms	4.21 ms	7.39 ms	0
256	100,000	26,684	3.06 ms	16.57 ms	24.35 ms	0

Cross-check against the broker's own metrics: after the three runs, the Prometheus endpoint reported `goqueue_messages_published_total 300000`, exactly matching ghz's $3 \times 100,000$ successful responses. The server-side latency histogram shows 99.98% of publishes handled in under 5 ms. The client tool and the server agree on every message.

Why ghz reads lower than the internal gRPC sweep (26.7k vs 83.9k): ghz measures full unary round-trips from a separate process through a general-purpose client, and at these rates the load generator itself becomes part of the measurement. That is exactly why both numbers belong in the story: the internal harness isolates broker capacity; ghz proves an outside observer sees healthy behavior with zero errors. This run used the broker's default WAL mode (`wal-sync-mode=none`, fsync deferred); durable-write throughput is what Section 3.2 measures.

Gap analysis: famous OSS testing tools, used and missing

The honest answer to "why is nothing famous in the stack": the interesting numbers (group commit, fsync batching, quorum commit) live inside the process where external tools cannot see, so a custom harness was the right first tool. But external tools add independence and comparability, and most are cheap to adopt.

TOOL	WHAT IT WOULD PROVE	STATUS	EFFORT
ghz (gRPC load)	Independent throughput/latency witness against a live broker	● DONE, this report	–
porcupine (linearizability checker)	Upgrades "kill tests pass" to "operation history is provably linearizable". Highest credibility-per-effort in this list; used by etcd and TiDB tests	MISSING, top priority	~2-3 days
GitHub Actions CI	Every number above re-verified on every push, badge in README	MISSING	~half a day
toxiproxy (network chaos)	Partition and latency injection between real nodes: the class of failure the in-process kill tests cannot express	MISSING	~2 days
go fuzzing (native)	Protocol frame parser and WAL record decoder survive malformed input	MISSING	~1 day
k6 (load scripting)	Scenario-shaped load (ramp, soak). Overlaps ghz for this protocol; optional	optional	~1 day
Jepsen	The gold standard, but a Clojure harness sized for commercial databases. porcupine + toxiproxy capture most of its value here at 5% of the cost	not worth it yet	weeks

Recommended order: porcupine first (it converts existing chaos tests into a formal correctness claim), then CI so every claim is continuously re-proven, then toxiproxy partitions. That sequence turns the README's story from "benchmarked by its author" into "verified by neutral, recognizable tools on every commit".

SECTION 6

Methodology and environment

Date	2026-07-12
Machine	AMD Ryzen AI 7 350, 8 cores / 16 threads, 15 GB RAM, Windows 11 Home
Toolchain	Go 1.26.1 (windows/amd64), ghz latest via <code>go install</code>
Tree state	main @ <code>7760532</code> plus uncommitted WIP: gossip advertise-address resolution in <code>membership.go</code> and matching <code>deploy/cluster.yml</code> binds. All results in this report include that WIP
Network	Loopback only. Cross-machine numbers will be lower; relative comparisons (A/B, sweeps) remain valid

Reproduce every number

```
# correctness
go test ./... -count=1
go vet ./...
go test -race -count=1 ./internal/cluster/... ./internal/wal ./internal/broker ./internal/controlplane

# integration + failover
GOQUEUE_BENCH=1 go test ./internal/cluster -tags cluster_integration -count=1 -v \
    -run 'TestZeroDataLossOnLeaderKill|TestThreeNodeCluster'

# internal benchmarks
GOQUEUE_BENCH=1 go test ./internal/cluster/shardwal -run TestShardGroupCommitReport -count=1 -v
GOQUEUE_BENCH=1 go test ./internal/cluster -tags cluster_integration -run TestClusterConcurrentThroughp
GOQUEUE_BENCH=1 go test ./bench -run 'TestTCPThroughputReport|TestLatencyReport|TestWALGroupCommitLaten

# external load test
go build -o broker.exe ./cmd/broker
./broker.exe --grpc-addr=127.0.0.1:19095 --metrics-addr=127.0.0.1:12112 &
ghz --insecure --proto proto/goqueue.proto --call goqueue.v1.BrokerService.Publish \
    -d '{"topic":"ghz-bench","payload":"<base64 256B>","key":"k1"}' \
    -c 256 -n 100000 --connections=8 127.0.0.1:19095
curl -s localhost:12112/metrics | grep goqueue_messages_published_total
```

AgentBus full testing report · generated 2026-07-12 · all tests executed live on the machine above; no numbers reused from prior runs.